PROJECT REPORT · FMCG SUPPLY CHAIN ANALYTICS

End-to-End Supply Chain Data Automation & Analytics

Gmail → n8n → Supabase (PostgreSQL) → Quadratic AI

Prepared by: Abhishek Shinde · October 2026

Project at a glance

Item	Details
Domain	FMCG supply chain — order delivery performance (On-Time, In-Full, OTIF)
Goal	Automatically move supply-chain CSV data from email into a cloud database, then analyse it with an AI spreadsheet to measure delivery reliability
Tools	Gmail & Gmail API, Google Cloud Console (OAuth 2.0), Node.js / npm, n8n (self-hosted on localhost), Supabase (PostgreSQL), Quadratic AI (Python), Open Exchange Rates API
Data model	5 tables — 2 fact tables (fact_order_line, fact_aggregate) and 3 dimension tables (dim_customers, dim_products, dim_targets_orders)
Workflow result	Workflow ran successfully end to end: 1 email triggered two parallel branches that extracted and inserted **57** and **109** rows into PostgreSQL (166 rows in total)
Reference	Codebasics Resume Project Challenge (FMCG, C2) step-by-step guides and a YouTube walkthrough

1. Project Overview

Supply chain teams in FMCG companies need to know, every day, whether customer orders are being delivered **on time** and **in full**. In practice, the raw order data often arrives as CSV files over email, gets downloaded by hand, and is pasted into spreadsheets — a slow and error-prone routine.

In this project I built a small but complete data pipeline that removes that manual work. When an email carrying the order data arrives in Gmail, an **n8n** workflow picks it up, reads the CSV attachments, and writes every row into a **PostgreSQL** database hosted on **Supabase**. The data is then analysed in **Quadratic AI**, an AI-powered spreadsheet, using natural-language prompts that generate Python code to clean the data, convert currencies, and compute the core supply-chain KPIs.

Objectives

- Automate data ingestion from email to database with no manual download or upload.
- Design a clean star-schema-style data model in PostgreSQL (facts + dimensions).
- Securely connect n8n to Gmail through a Google Cloud OAuth 2.0 client.
- Use AI-assisted analysis to calculate Line Fill Rate, Volume Fill Rate, OT %, IF % and OTIF %.
- Answer business questions on revenue loss, customer performance and product bottlenecks.

Solution architecture

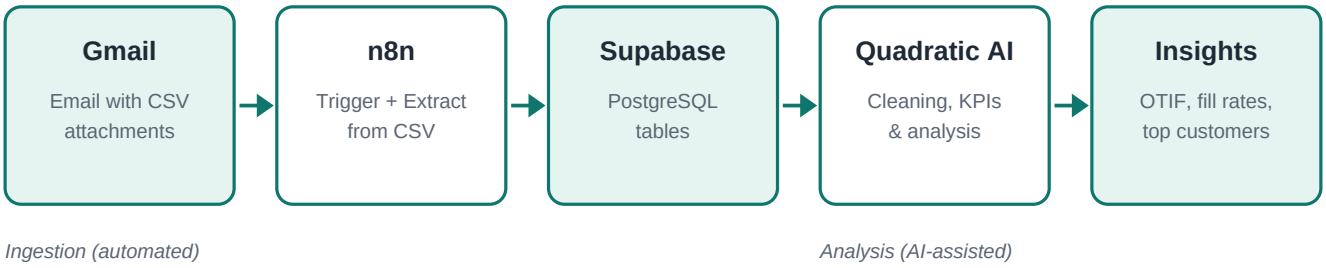


Figure 1 — End-to-end data flow of the project

2. Tools & Technologies Used

Tool	What it is	How I used it in this project
Node.js & npm	JavaScript runtime and package manager	Installed with the Windows .msi installer (including the optional build tools / Chocolatey) so that n8n could be installed through npm.
n8n	Open-source workflow automation platform	Installed globally (`npm install -g n8n`) and run on localhost:5678. Built the Gmail → Extract from File → Postgres workflow.
Google Cloud Console	Google's developer platform	Created a project, enabled the Gmail API, configured the OAuth consent screen and created a Web-application OAuth Client ID for n8n.
Gmail / Gmail API	Email service and its programmatic interface	Source of the data: the Gmail Trigger node watches the inbox for the email carrying the CSV attachments.
Supabase	Hosted Postgres platform (open-source Firebase alternative)	Created the organisation and project, built five tables by importing CSVs, fixed date column types and copied the connection parameters (Session Pooler) for n8n.

Tool	What it is	How I used it in this project
PostgreSQL	Relational database	Storage layer for the fact and dimension tables used in analysis.
Quadratic AI	AI-powered spreadsheet with built-in Python	Used prompts to generate a date table, an exchange-rate table, a cleaned fact_summary table and all KPIs.
Open Exchange Rates API	Historical currency rates API	Fetched daily USD → INR rates so that USD-priced orders could be valued in INR.

3. What I Did — Step by Step

Step 1 · Set up n8n on localhost

- Downloaded and installed Node.js using the Windows Installer (.msi), accepting the licence and ticking *"Automatically install the necessary tools"* (this also installs Chocolatey).
- Opened Command Prompt as Administrator and installed n8n globally:

```
npm install -g n8n
n8n    # starts the server; press "o" to open it in the browser
```

- Created the n8n owner account and completed the onboarding questions. n8n was then available at `http://localhost:5678`.

Step 2 · Create the PostgreSQL database in Supabase

- Signed up on Supabase, confirmed the email, created an organisation and a new project with a strong project-level database password.
- In the Database section, created each table with **"Import data from CSV"** using the provided input files: fact_aggregate, dim_customers, dim_products, dim_targets_orders and fact_order_line.
- Updated date columns (e.g. order_placement_date in fact_aggregate) to the **date** data type.
- Opened **Connect** and chose the Session Pooler connection (recommended for IPv4 networks) to get host, port, database, user and password for n8n.

Step 3 · Create an OAuth Client ID for the Gmail API

- In Google Cloud Console created a new project and enabled the **Gmail API** from APIs & Services.
- Configured the OAuth consent screen (app name, support email, audience = External, contact email) and added my Gmail address as a test user.
- Created an OAuth client of type **Web application** and added n8n's redirect URI:

```
http://localhost:5678/rest/oauth2-credential/callback
```

- Copied the Client ID and Client Secret into n8n's Gmail OAuth2 credential, signed in with Google, granted all requested scopes, and got *"Account connected successfully."*

Step 4 · Build the n8n automation workflow

The workflow has five nodes and two parallel branches — one per CSV attachment:

Node	Type / operation	Purpose
Gmail Trigger	Gmail Trigger (with attachments)	Fires when the data email arrives and passes the message with its binary attachments.

Node	Type / operation	Purpose
Extract from File	Extract From CSV	Parses the first CSV attachment into rows (57 items).
Extract from File1	Extract From CSV	Parses the second CSV attachment into rows (109 items).
Insert rows in a table	Postgres · insert	Maps the 57 rows to the target table in Supabase and inserts them.
Insert rows in a table1	Postgres · insert	Maps the 109 rows to the second target table and inserts them.

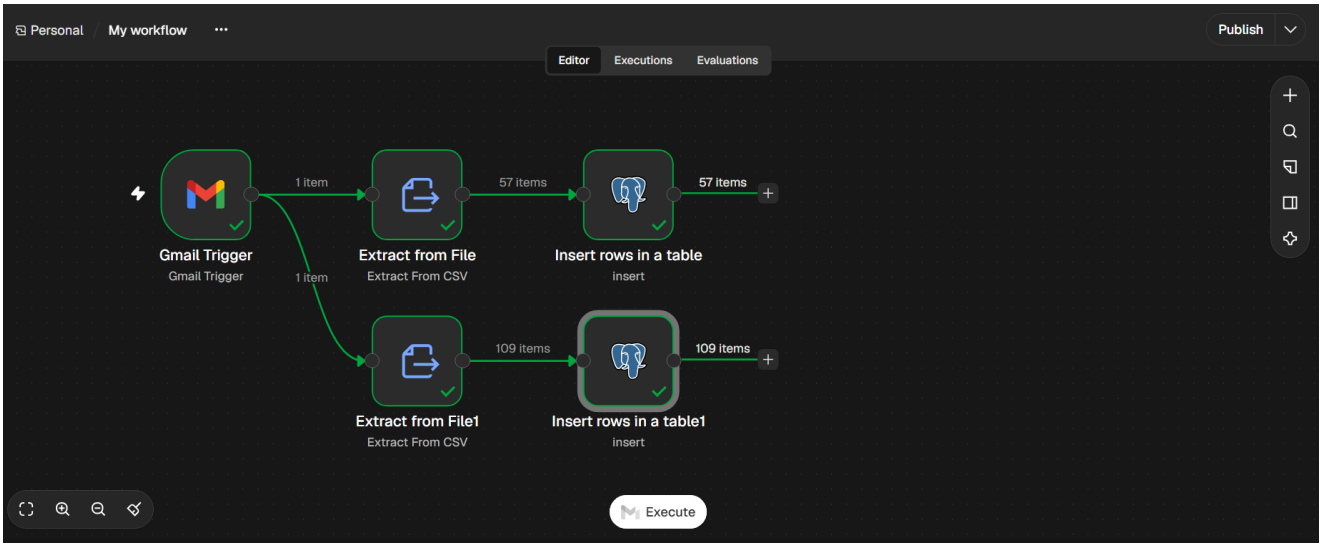


Figure 2 — My n8n workflow after a successful execution (all nodes show green ticks)

Step 5 · Analyse the data in Quadratic AI

With the data in PostgreSQL, I connected it to Quadratic AI and used a sequence of prompts. Each prompt makes Quadratic generate and run Python code inside the sheet:

#	Prompt goal	What it produces
1	Date table	A calendar of every date from 1 Mar 2025 to 31 May 2025.
2	Exchange-rate table	Daily USD → INR rates (1 Mar – 17 May 2025) pulled from the Open Exchange Rates historical API, one request per date.
3	Clean & summarise (fact_summary)	Loads orders, products, customers and rates; converts and validates IDs; drops NULL IDs; parses dates; joins everything; computes **total_amount** in INR (USD: price_USD × rate × qty, INR: price_INR × qty); removes helper columns.
4	Business KPIs	Total Order Lines, Line Fill Rate, Volume Fill Rate, Total Orders, On-Time %, In-Full %, OTIF %.
5	Top customers	Top 5 customers by order value with name, ID, city and their OTIF %, IF %, OT %.
6	Top customers (India)	The same view filtered to customers in India.

Connecting Quadratic to the Supabase database

In Quadratic I created a file named **Customer Data** and added a PostgreSQL connection called **n8n proj**, pointing to the same Supabase database that n8n writes into. Through this connection Quadratic can browse the database schema and run SQL directly inside spreadsheet cells.

- Created one sheet per table: **dim_customers**, **dim_products**, **dim_targets_orders**, **fact_order_line** and **fact_aggregate**.

- Each sheet holds an SQL code cell that pulls its table live from Postgres, for example:

```
SELECT * FROM "public"."fact_aggregate" LIMIT 100
```

- The query result spills into the grid as a formatted table that refreshes on re-run (the fact_aggregate query returned a 6 × 100 array in 23 ms).
- The **Dim_Products** table shows the product catalogue: 18 AM-branded products across **Dairy** (milk, butter, ghee, curd), **Food** (biscuits) and **beverages** (tea), each priced in INR and USD — the inputs used later for order-value calculation.

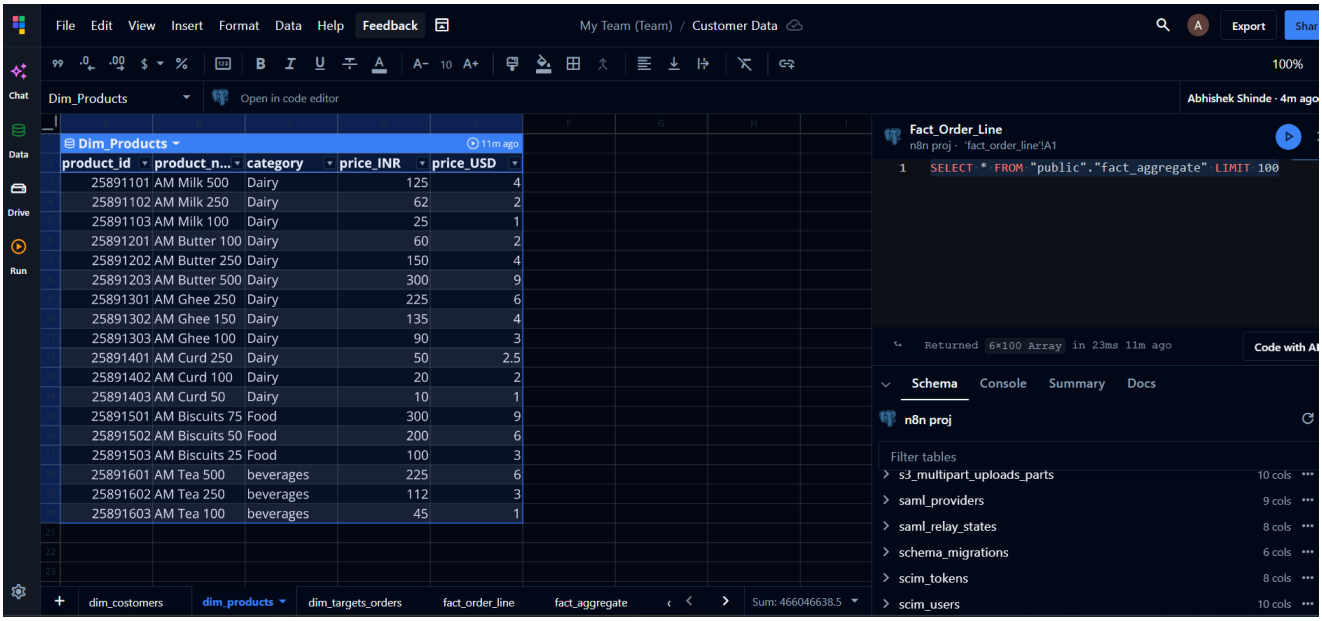


Figure 3 — Quadratic AI file “Customer Data”: the Dim_Products table loaded from Supabase, with the SQL code editor and the n8n proj connection schema on the right

4. Database Design (Supabase · schema public)

The database follows a star-schema pattern: fact tables hold measurable events (order lines and order-level results), and dimension tables describe customers, products and targets. They join on `customer_id`, `product_id` and `order_id`.

Table	Type	Columns (data type)
fact_order_line	Fact (line level)	order_id (text), order_placement_date (text), customer_id (int8), product_id (int8), order_qty (int8), agreed_delivery_date (text), actual_delivery_date (text), delivery_qty (int8), In Full (int8), On Time (int8), On Time In Full (int8)
fact_aggregate	Fact (order level)	order_id (text), customer_id (int8), order_placement_date (date), on_time (int8), in_full (int8), otif (int8)
dim_customers	Dimension	customer_id (int8), customer_name (text), city (text), currency (text)
dim_products	Dimension	product_name (text), product_id (int8), category (text), price_INR (int8), price_USD (float8)
dim_targets_orders	Dimension	customer_id (int8), ontime_target% (int8), infull_target% (int8), otif_target% (int8)

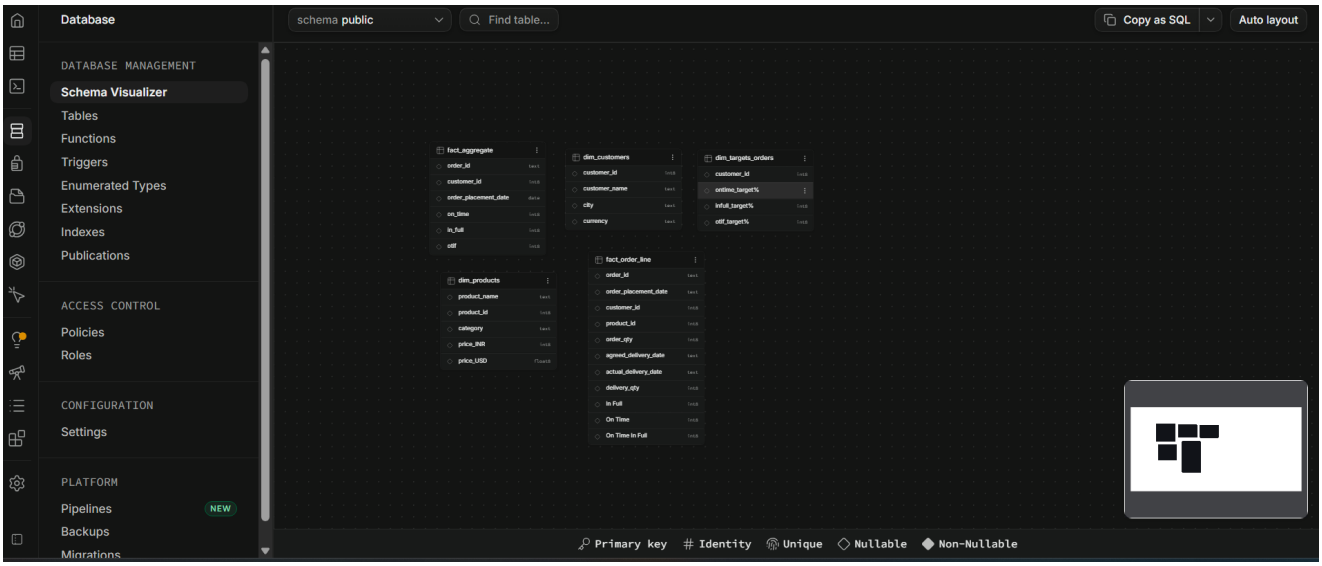


Figure 4 — Supabase Schema Visualizer showing the five tables

5. Supply Chain Concepts & KPIs

Order vs. order line. An order is a unique request placed by a customer on a given date. Each separate product inside that order is an *order line*. Example: ordering 4 notebooks and 2 pens is one order with two lines.

KPI	Level	Formula / rule	Who uses it
Line Fill Rate	Line	Lines shipped in full ÷ total lines ordered (ignores timing)	Supply planning
Volume Fill Rate	Line	Total quantity shipped ÷ total quantity ordered	Supply planning
On-Time Delivery %	Order	Orders where *every* line arrived by the agreed date ÷ total orders	Warehouse & distribution
In-Full Delivery %	Order	Orders where *every* line was delivered in the full quantity ÷ total orders	Supply planning
OTIF %	Order	Orders that are both On Time **and** In Full ÷ total orders — the strictest measure of reliability from the customer's view	Entire supply chain

Worked example: a customer orders 4 notebooks and 2 pens; 4 notebooks and 1 pen are shipped. Line Fill Rate = 1 ÷ 2 = **50%** (the pen line failed). Volume Fill Rate = 5 ÷ 6 = **83%**. The order is not In Full, so it also fails OTIF.

Business questions explored

- How much revenue is lost because of undelivered quantities?
- Which customers show the largest gaps between their OTIF target and actual OTIF?
- Which product categories have the lowest In-Full delivery rates — and what does that say about supply bottlenecks?
- What is the average delay (in days) for late deliveries?
- Who are the top 5 customers by order value, overall and within India, and how reliably are they served?

6. Output & Results

Automation output

Stage	Result
Gmail Trigger	1 item — the incoming email with its CSV attachments was captured
Branch 1	Extract from File → **57 items** parsed → **57 rows inserted** into PostgreSQL
Branch 2	Extract from File1 → **109 items** parsed → **109 rows inserted** into PostgreSQL
Total	**166 rows** loaded automatically; every node completed successfully

Database output

A working PostgreSQL database on Supabase with five related tables (Figure 4), correct data types for dates, and pooled connection details that external tools such as n8n and Quadratic can use.

Analysis output

- **Quadratic "Customer Data" file** — five sheets fed live from Supabase through the n8n proj connection (Figure 3).
- **fact_summary** — one clean, analysis-ready table with all order values in INR.
- Supporting **date** and **exchange_rate** tables for time-based and currency analysis.
- A KPI summary: Total Order Lines, Line Fill Rate, Volume Fill Rate, Total Orders, OT %, IF % and OTIF %.
- Top-5 customer tables (overall and India) with order value and OT / IF / OTIF %.

7. Key Learnings

- **Workflow automation:** how triggers, binary attachments and data-transformation nodes fit together in n8n, and how to run parallel branches from a single trigger.
- **Authentication:** setting up OAuth 2.0 end to end — consent screen, test users, client ID/secret and redirect URIs.
- **Cloud databases:** creating Postgres tables from CSV, fixing data types, and using a connection pooler for IPv4 networks.
- **Data modelling:** separating facts and dimensions so KPIs can be sliced by customer, product, city and date.
- **AI-assisted analytics:** writing precise prompts that turn into reliable Python for cleaning, joining, currency conversion and KPI calculation.
- **Domain knowledge:** the difference between line-level and order-level metrics, and why OTIF is the toughest reliability measure.

8. Possible Next Steps

- Add a de-duplication / upsert step so re-sent emails don't create duplicate rows.
- Filter the Gmail Trigger by sender or subject and add an error-notification branch.

- Schedule a daily KPI email or Slack summary generated straight from the database.
- Build a live dashboard on top of Supabase tracking OTIF against customer targets.

9. References

- Codebasics Resume Project Challenge (FMCG | C2) — Supply Chain Concepts, Quadratic exercises and Required Prompts guides.
- Setup guides: 1 · n8n Setup in Local Host; 2 · Set up a Postgres DB in Supabase; 3 · Creating an OAuth Client ID for the Gmail API.
- YouTube project walkthrough used as the reference for this build.
- Tools: n8n.io · supabase.com · quadratichq.com · console.cloud.google.com · openexchangerates.org